

Relatório de Comparação de Algoritmos de Ordenação

Renato Venâncio de Almeida, Thiago Lopes Coelho

FASEH - Faculdade da Saúde e Ecologia Humana

Curso de Ciências da Computação

Vespasiano – MG, Brasil

972410267@ulife.com.br, 972410118@ulife.com.br

Resumo

Este trabalho apresenta uma análise comparativa de diferentes algoritmos de ordenação desenvolvidos na linguagem Java. Para isso, foram conduzidos testes utilizando conjuntos de dados de diferentes dimensões. Com base nos resultados, foi possível compreender melhor as características de cada algoritmo e identificar suas vantagens e desvantagens em diferentes cenários de processamento e organização de dados.

Palavras-chave

Este trabalho tem como objetivo comparar o desempenho dos algoritmos Bubble Sort e Quick Sort utilizando conjuntos de dados de diferentes tamanhos e características, analisando o tempo de execução de cada método.

Abstract

This work presents a comparative analysis of different sorting algorithms developed in the Java language. To this end, tests were conducted using datasets of different dimensions. Based on the results, it was possible to better understand the characteristics of each algorithm and identify their advantages and disadvantages in different data processing and organization scenarios.

Keywords

This work aims to compare the performance of the Bubble Sort and Quick Sort algorithms using datasets of different sizes and characteristics, analyzing the execution time of each method.

1. Introdução

Os algoritmos de ordenação são amplamente utilizados para organizar informações e facilitar o processamento de dados em sistemas computacionais.

Entre os algoritmos mais populares estão o Bubble Sort e o Quick Sort, que possuem características e níveis de eficiência diferentes.

O objetivo deste trabalho é comparar o desempenho desses dois métodos por meio de testes realizados com conjuntos de dados de diferentes tamanhos. Com base nos

resultados obtidos, pretende-se analisar as vantagens e desvantagens de cada algoritmo e compreender sua aplicação em diferentes situações

2. Metodologia

Foram realizados testes com conjuntos de 1.000 e 1.000.000 de elementos, considerando três cenários: dados aleatórios, dados ordenados e dados ordenados inversamente.

O tempo de execução foi medido utilizando o método `System.currentTimeMillis()`.

2.1 Ambiente de Teste

Os experimentos foram realizados em um computador equipado com processador Intel Core i5 12400F, 16 GB DDR4 de memória RAM e sistema operacional Windows 11.

Os algoritmos foram implementados na linguagem Java utilizando o JDK 25. A execução e os testes foram realizados por meio da IDE IntelliJ IDEA.

Foram utilizados conjuntos de dados de diferentes tamanhos para avaliar o desempenho dos algoritmos Bubble Sort e Quick Sort. O tempo de execução foi medido em milissegundos, permitindo a comparação da eficiência de cada algoritmo em diferentes cenários.

3. Resultados

Tabela 1. Tempos de execução dos algoritmos de ordenação

Tamanho	Algoritmo	Melhor Caso (ms)	Caso Médio (ms)	Pior Caso (ms)
1.000	Bubble Sort	0	3	3
1.000	Quick Sort	4	0	2
1.000.000	Bubble Sort	76094	Não concluído	591227
1.000.000	Quick Sort	StackOverflowError	2	StackOverflowError

3.1 Análise de Resultados

O Bubble Sort apresentou bom desempenho apenas para conjuntos pequenos, tornando-se muito lento para grandes volumes de dados devido à sua complexidade $O(n^2)$.

O Quick Sort foi significativamente mais rápido nos dados aleatórios. Entretanto, nos conjuntos já ordenados e ordenados inversamente ocorreu `StackOverflowError` devido ao excesso de chamadas recursivas causado pela escolha do último elemento como pivô.

4. Conclusão

Com base nos testes, verificou-se que o Quick Sort foi o algoritmo mais eficiente, apresentando tempos de execução menores, especialmente em conjuntos de dados maiores.

Mesmo que seu desempenho possa variar de acordo com a escolha do pivô, ele ainda se mostrou mais produtivo que o Bubble Sort. Em contrapartida, o Bubble Sort mostrou um desempenho satisfatório apenas em conjuntos pequenos. Desse modo, conclui-se que o Quick Sort é a opção mais indicada quando se busca rapidez e eficiência na ordenação de dados.

5. Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C.

Introduction to Algorithms. Cambridge: MIT Press.

ORACLE. Java Documentation. Disponível em: <https://docs.oracle.com/>.

GEEKSFORGEEEKS.

Bubble Sort Algorithm. Disponível em: <https://www.geeksforgeeks.org/>.

GEEKSFORGEEEKS.

Quick Sort Algorithm. Disponível em: <https://www.geeksforgeeks.org/>.

OPENDSA. Quicksort and Bubble Sort. Disponível em: <https://opensa-server.cs.vt.edu/>.